Assignment 04

Published | Assign To | Edit

AWS Organization Setup

Overview

You'll create an AWS Organization with three accounts: one root account and two member accounts (dev and demo).

Important Note: The email addresses shown are examples only. You may use any email provider (Gmail is not required).

Steps

1. Enable AWS Organizations

- Log in to your existing AWS account - this will become your **root account**
- Navigate to the AWS Organizations service
- Enable Organizations for your account

2. Create the Dev Member Account

- **Purpose:** This account is for your development work while completing assignments
- **Email requirement:** You need a unique email address
  - **Gmail tip:** If using Gmail, you can create email aliases by adding `+dev` before the `@` symbol
  - **Example:** `j.doe@gmail.com` → `j.doe+dev@gmail.com`
  - These aliases deliver to your main inbox but count as separate email addresses in AWS

3. Create the Demo Member Account

- **Purpose:** This account will be used to demonstrate your completed assignments for grading
- **Email requirement:** You need another unique email address (different from dev)
  - **Gmail tip:** Use a different alias, like `+demo`
  - **Example:** `j.doe@gmail.com` → `j.doe+demo@gmail.com`

AWS IAM Setup

Security Requirement

Critical: Only you should have permission to modify (create, update, delete) resources in your environment. Teaching assistants should only be able to view resources.

Where to Perform These Steps

⚠️ **Complete all IAM setup in your DEMO account** (not root or dev)

Task 1: Create IAM Group

Group Name: `csye6225-ta`

Purpose: This group will contain all teaching assistant user accounts

Permissions: Read-only access to all AWS services

Steps:

1. Navigate to IAM → Groups
2. Create a new group named `csye6225-ta`
3. Attach the AWS managed policy: `ReadOnlyAccess`
   - Policy ARN: `arn:aws:iam::aws:policy/ReadOnlyAccess`
   - This is a pre-defined AWS policy - search for it rather than creating a new one

Task 2: Create IAM Users

Who to create accounts for:

- ✅ Create user accounts for **all teaching assistants**
- ❌ Do **NOT** create an account for the instructor

User Configuration:

- **Username:** Use each TA's first name
- **Password:** Leave "Autogenerated password" checked (default setting)
- **Access credentials:** Do NOT configure access keys or other credentials
- **Group membership:** Add each user to the `csye6225-ta` group

Important - Password Distribution: ⚠️ AWS does NOT automatically email autogenerated passwords to users. You must:

1. Copy each generated password when displayed
2. Manually email each password to the corresponding teaching assistant

Automating Application Setup with Shell Script

Overview

Create a shell script that automates the complete setup of your application environment on a fresh Ubuntu 24.04 LTS system.

Objective

Write a shell script that will prepare a new Ubuntu server to run your application by installing dependencies, configuring the database, setting up user accounts, and deploying your application files.

Script Requirements

Your shell script must perform the following tasks **in order**:

1. Update Package Lists

- Run the command to refresh the package repository information
- This ensures you'll install the latest available versions of packages
- **Command hint:** `apt update`

2. Upgrade System Packages

- Update all installed packages to their latest versions
- This ensures the system has the latest security patches and features
- **Command hint:** `apt upgrade`

3. Install Database Management System

- Install **ONE** of the following RDBMS options:
  - MySQL, **OR**
  - PostgreSQL, **OR**
  - MariaDB
- Choose the database system that your application uses
- Ensure the database service starts and is enabled on boot

4. Create Application Database

- Create a new database for your application to use
- The database name should be appropriate for your application (e.g., `csye6225_db`)
- You may also need to create a database user with appropriate permissions

5. Create Application Linux Group

- Create a dedicated Linux group for running your application
- **Suggested name:** `csye6225` or similar
- This group will be used for managing file permissions

6. Create Application User Account

- Create a dedicated Linux user account for running your application
- This user should be part of the group created in step 5

- **Best practice:** Applications should not run as root user
- Consider using a system user account (non-login user)

7. Deploy Application Files

- Unzip/extract your application files to: `/opt/csye6225/`
- This directory should be created if it doesn't exist
- All application code, configuration files, and dependencies should be placed here

8. Set File Permissions

- Update the ownership of `/opt/csye6225/` and all its contents
- The application user and group (created in steps 5-6) should own these files
- Set appropriate read/write/execute permissions
- Ensure the application can run, but unnecessary write permissions are restricted

---

Additional Requirements

Script Location

- Add this shell script to **your web application's Git repository**
- **Suggested location:** Place it in the root directory or in a `scripts/` folder
- **Suggested filename:** `setup.sh` or `install.sh`

Script Best Practices

Your script should:

- Include a shebang line at the top (e.g., `#!/bin/bash`)
- Include comments explaining each major section
- Handle errors appropriately (consider using `set -e` to exit on errors)
- Run with sudo/root privileges (or check for them)
- Be idempotent when possible (safe to run multiple times)

Testing Your Script

Before submission, verify that:

1. The script runs without errors on a clean Ubuntu 24.04 LTS system
2. All packages are successfully installed
3. The database is created and accessible
4. The user and group exist with correct permissions
5. Your application files are in `/opt/csye6225/`
6. File permissions are correctly set
7. Your application can start and run successfully after the script completes

Install and Configure the AWS Command Line Interface

1. **Install the AWS CLI** on your development machine (laptop)
   - Follow the official installation guide: **Install the AWS Command Line Interface on Linux (https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html)**
   - Verify installation by running: `aws --version`
2. **Configure AWS CLI profiles** (do NOT configure a `default` profile)
   - Create a `dev` profile for your development AWS account:

```
aws configure --profile dev
```

- Create a `demo` profile for your demonstration AWS account:

```
aws configure --profile demo
```

3. **Set the AWS region** for both profiles
   - During configuration, set the default region to the AWS region geographically closest to your location
   - Common regions: `us-east-1` (N. Virginia), `us-west-2` (Oregon), `eu-west-1` (Ireland), `ap-southeast-1` (Singapore)
   - You can verify your configuration in `~/.aws/config` and `~/.aws/credentials`

Important: Do not create or use a `default` AWS CLI profile. Always specify either `--profile dev` or `--profile demo` when running AWS CLI commands.

Infrastructure as Code

Important: Use unique names for all resources to support multiple deployments in the same AWS account.

Objective: Set up AWS networking infrastructure (VPC, Internet Gateway, Route Table, Routes) using Terraform.

GitHub Repository Setup

1. **Create repository**
   - Create a **private** repository named `tf-aws-infra` in your GitHub organization
   - Repository link: `https://github.com/<your-org>/tf-aws-infra`
2. **Initialize repository**
   - Add a `README.md` with Terraform setup instructions
   - Add a `.gitignore` file (use **GitHub's Terraform template (https://github.com/github/gitignore/blob/main/Terraform.gitignore)**)

3. **Development workflow**
   - Fork the repository to your personal namespace
   - Perform all development work in your fork
   - Store all Terraform templates (`.tf` files) in this repository

GitHub Repository Branch Protection Rules

Requirement: Implement branch protection rules for **all** repositories (past, present, and future) created in this course.

Configuration Steps

1. **Navigate to branch protection settings**
   - Go to repository **Settings** → **Branches** → **Add branch protection rule**
2. **Configure protection rules** for the default branch (typically `main` or `master`)
   - **Branch name pattern:** `main` (or your default branch name)
   - **Required settings:**
     - ☑ Require a pull request before merging
     - ☑ Dismiss stale pull request approvals when new commits are pushed
     - ☑ Require status checks to pass before merging (configure after CI/CD setup)
     - ☑ Require branches to be up to date before merging
     - ☑ Do not allow bypassing the above settings
3. **Apply to all repositories**
   - Repeat this configuration for every course-related repository
   - Apply these rules to any new repositories you create

Reference: **GitHub Protected Branches Documentation (https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches)**

AWS Networking Setup

Here is what you need to do for networking infrastructure setup:

1. Create a **Virtual Private Cloud (VPC) (https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html)** .
2. Create **subnets (https://docs.aws.amazon.com/vpc/latest/userguide/working-with-vpcs.html#AddaSubnet)** in your VPC. You must create `3 public subnets` and `3 private subnets` , each in a different availability zone in the same region in the same VPC. Each of the `3` availability zones must have one public and one private subnet.
3. Create an **Internet Gateway (https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html)** resource and attach

the Internet Gateway to the VPC.
4. Create a **public route table (https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html)** . Attach all public subnets created to the route table.
5. Create a **private route table (https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html)** . Attach all private subnets created to the route table.
6. Create a `public` route in the `public route table` created above with the destination CIDR block `0.0.0.0/0` and the internet gateway created above as the target.

Infrastructure as Code with Terraform

For this objective, you must complete the following tasks:

1. Install and set up the AWS command-line interface.
2. Create a Terraform configuration file that will set up all the networking resources.
3. Values should not be hard-coded in your Terraform configuration files.
4. You must be able to use the same Terraform configuration files in the same AWS account and region to create multiple VPCs, including all of its resources (listed in the “AWS Networking Setup” section), such as subnets, internet gateway, route table, etc.

Continuous Integration (CI)

Implement Continuous Integration (CI) with GitHub Actions for IaC Repository

1. Add a GitHub Actions workflow to run the ***terraform fmt* (https://developer.hashicorp.com/terraform/cli/commands/fmt)** (recursively) and ***terraform validate* (https://developer.hashicorp.com/terraform/cli/commands/validate)** command for each pull request raised. A pull request can only be merged if the workflow executes successfully.
2. Add **Status Checks GitHub branch protection (https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches)** to prevent users from merging a pull request when the GitHub Actions workflow run `fails`.

Assignment Submission Instructions

Development Workflow- Branch Management

- Complete all work on a feature branch in your forked repository

- Merge your feature branch to `main` only when development is complete
- Ensure your feature branch and `main` branch are synchronized before submission

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input

- `account_updated` field: Set automatically by application, never by user input
- Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)

- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class`, `.jar`, `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/`, `dist/`, `target/` directories
- **System Files**: No OS-specific files (`.DS_Store`, `Thumbs.db`, etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

AWS IAM Verification

- Root account has Organizations enabled
- Dev member account created with a unique email
- Demo member account created with a unique email
- IAM group `csye6225-ta` created in DEMO account
- ReadOnlyAccess policy attached to the group
- IAM user accounts created for all TAs (first names only)
- All TA users added to the `csye6225-ta` group
- Autogenerated passwords saved and emailed to each TA
- No IAM user account created for the instructor
- Verify the user does not have console access and has only programmatic access.
- Verify that a new IAM user has been created for the GitHub Actions workflow.

Infrastructure as Code with Terraform

All demos must be performed by downloading code from the student's GitHub repository's main branch through the browser.

- Students should demo the creation of networking resources using the `terraform apply` command.
- Students should demo the cleanup of networking resources using the `terraform destroy` command.
- Verify that the Terraform template does not contain any hardcoded values.
- Create multiple VPCs (and resources) without any conflicts in the same AWS account & same region.
- Create other VPCs (and resources) without any conflicts in different AWS regions.

Continuous Integration

- Branch protection is enabled for all the repositories created for the course.
- GitHub Workflows prevent pull requests from being merged if the code fails to compile or if the test fails for the web app repository. Students will demonstrate this by raising a pull request that causes

the test to fail.
- GitHub Workflows prevent pull requests from being merged if the Terraform code is not formatted correctly or if the validation fails.

Automating Application Setup with Shell Script

- Shell script created with all the required steps
- Script includes a proper shebang line
- Script is commented for clarity
- Script handles errors appropriately
- Script added to web app repository
- Script tested on Ubuntu 24.04 LTS
- All steps executed successfully
- Application runs correctly after script execution

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Oct 9 at 3pm	Everyone	Oct 2 at 6pm	Oct 10 at 3pm

+ **Rubric**